DIITT | ECYS

DESARROLLO DE TRANSFERENCIA TECNOLÓGICA

Teoría de autómatas, lenguajes y computación

# MANUAL TECNICO

Proyecto 1 – Laboratorio Leguajes formales de computación.


Nombre: Bily Estuardo Vallecidos Folgar

Carnet: 202401753

Auxiliar: Rivers


USAC TRICENTENARIA Universidad de San Carlos de Guatemala

# INTRODUCCION:

MedLexer es una aplicación de escritorio desarrollada en C++17 utilizando el framework Qt 6.8.3, diseñada para el análisis léxico de archivos con extensión .med escritos en el mini-lenguaje formal MedLang. El sistema implementa manualmente un Autómata Finito Determinista (AFD) sin el uso de herramientas automáticas de generación léxica ni expresiones regulares, cumpliendo estrictamente con los principios teóricos de la teoría de autómatas y lenguajes formales.

El propósito principal de MedLexer es simular el procesamiento léxico de datos hospitalarios estructurados, demostrando cómo un compilador real procesa y valida un lenguaje de dominio específico. La herramienta es capaz de reconocer, clasificar y validar más de 30 tipos de tokens distintos, detectar y reportar errores léxicos con precisión de línea y columna, y generar cuatro reportes HTML interactivos junto con un diagrama de relaciones en formato Graphviz DOT.

## Justificación Técnica

El núcleo del sistema es la clase LexicalAnalyzer, la cual implementa el AFD mediante una función de transición de estados explícita usando la instrucción switch(state) de C++. Esta decisión de diseño se tomó por las siguientes razones técnicas:

Primero, cada case del switch corresponde directamente a un estado del AFD diseñado formalmente, lo que permite verificar la correspondencia entre el diagrama teórico y la implementación práctica de manera directa e inequívoca. Segundo, el compilador de C++ traduce el switch a una tabla de saltos en código máquina, garantizando tiempo de acceso O(1) por transición y haciendo el análisis eficiente incluso para archivos de hasta 1000 líneas. Tercero, agregar nuevos estados al autómata solo requiere agregar un nuevo case sin modificar la lógica de los estados existentes, lo que hace el sistema mantenible y extensible.

El método analyze() implementa el paradigma de lectura carácter por carácter usando dos funciones auxiliares fundamentales: peek() que retorna el carácter actual sin consumirlo, y advance() que lo consume y actualiza el contador de línea y columna. Este enfoque garantiza un tracking preciso de la posición de cada token en el archivo fuente, información crítica para el reporte de errores.

---

**Descripción del Lenguaje MedLang**

MedLang es un lenguaje de dominio específico diseñado para representar información hospitalaria estructurada. Su gramática léxica define los siguientes grupos de tokens:

**Palabras reservadas de estructura:** HOSPITAL, PACIENTES, MEDICOS, CITAS y DIAGNOSTICOS definen los bloques principales del archivo. Son sensibles a mayúsculas y deben escribirse exactamente como se definen.

**Palabras reservadas de elemento:** paciente, medico, cita y diagnostico inician la definición de cada entidad dentro de su bloque correspondiente. Son sensibles a minúsculas.

**Palabras reservadas de atributo:** edad, tipo_sangre, habitacion, especialidad, codigo, fecha, hora, con, condicion, medicamento y dosis definen los campos de cada entidad.

**Enumeraciones de especialidad:** CARDIOLOGIA, NEUROLOGIA, PEDIATRIA, CIRUGIA, MEDICINA_GENERAL y ONCOLOGIA son los únicos valores válidos para el campo especialidad de un médico.

**Enumeraciones de dosis:** DIARIA, CADA_8_HORAS, CADA_12_HORAS y SEMANAL son los únicos valores válidos para el campo dosis de un diagnóstico.

**Literales de fecha:** Siguen el formato estricto AAAA-MM-DD donde el mes debe estar entre 01 y 12, y el día entre 01 y 31. El AFD valida estos rangos durante la tokenización.

**Literales de hora:** Siguen el formato HH:MM en sistema de 24 horas donde la hora debe estar entre 00 y 23, y los minutos entre 00 y 59.

**Literales de cadena:** Texto entre comillas dobles. Si una cadena no se cierra antes del fin de línea o fin de archivo, se registra un error de gravedad CRÍTICO y se genera un token especial N/A - Error en cadena.

**Literales numéricos:** Números enteros positivos usados para edad y número de habitación.

**Delimitadores estructurales:** Los caracteres {, }, [, ], :, , y ; son tokens de tipo SYMBOL que definen la estructura jerárquica del archivo.

---

## Descripción del AFD

El autómata finito determinista implementado cuenta con 8 estados activos más estados de aceptación y error:

**Estado q0 (Inicial):** Es el estado de arranque y de retorno después de reconocer cada token. Desde aquí se decide el tipo de token a reconocer basándose en el primer carácter leído. Los espacios y saltos de línea se ignoran y se permanece en q0. Las letras y guiones bajos llevan a q1, los dígitos a q2, las comillas a q3, y los símbolos estructurales generan directamente un token SYMBOL.

**Estado q1 (Identificador):** Acumula caracteres alfanuméricos y guiones bajos. Al encontrar un carácter no alfanumérico, acepta el lexema acumulado y lo consulta en la tabla de keywords. Si el lexema coincide con una palabra reservada retorna su TokenType correspondiente, si no retorna UNKNOWN.

**Estado q2 (Número/Fecha/Hora):** Acumula dígitos. Si se han acumulado exactamente 4 dígitos y el siguiente carácter es un guión, transiciona a q4 para continuar leyendo una fecha. Si se han acumulado exactamente 2 dígitos y el siguiente carácter es dos puntos, transiciona a q7 para continuar leyendo una hora. En cualquier otro caso acepta como NUMBER.

**Estado q3 (Cadena):** Acumula cualquier carácter hasta encontrar una comilla de cierre, momento en que acepta como STRING. Si encuentra fin de archivo o salto de línea sin haber cerrado la cadena, registra un error CRÍTICO y genera un token especial.

**Estados q4 y q5 (Fecha MM y guión):** Leen los dos dígitos del mes y el segundo guión de una fecha respectivamente. Validan que el mes esté entre 01 y 12.

**Estado q6 (Fecha DD):** Lee los dos dígitos del día y valida que esté entre 01 y 31. Si es válido acepta como DATE, si no registra error.

**Estado q7 (Hora MM):** Lee los dos dígitos de los minutos y valida que la hora esté entre 00 y 23 y los minutos entre 00 y 59. Si es válido acepta como TIME, si no registra error.

---

## Descripción del Sistema de Reportes

El módulo ReportGenerator implementa un sistema de extracción de datos semánticos a partir de la secuencia de tokens generada por el analizador léxico. Este proceso se realiza en una segunda pasada sobre el vector de

tokens usando cuatro funciones de extracción especializadas: extractPatients, extractDoctors, extractAppointments y extractDiagnoses.

Cada función de extracción inicializa los campos de la estructura de datos con valores por defecto N/A antes de comenzar la lectura. Esto garantiza que si un atributo contiene un token inválido o está ausente por un error léxico previo, el objeto igual se incluye en el reporte con el campo marcado como N/A en lugar de descartarse completamente. Esta decisión es fundamental para sistemas hospitalarios reales donde perder un registro completo por un error de formato sería inaceptable.

Los cuatro reportes HTML generados usan CSS embebido y son completamente autocontenidos, lo que significa que pueden abrirse en cualquier navegador sin dependencias externas. Los colores semánticos usados en los reportes siguen una convención consistente: verde para estados normales o confirmados, naranja para advertencias o estados pendientes, y rojo para errores críticos o conflictos.

El Reporte 3 implementa detección automática de conflictos de horario comparando médico, fecha y hora para cada par de citas. Si el mismo médico tiene dos citas en la misma fecha y hora, ambas se marcan como CONFLICTO en rojo. El Reporte 4 calcula indicadores clave como promedio de edad, medicamento más prescrito y especialidad con mayor carga de citas mediante análisis cruzado de todos los bloques del archivo.

## Descripción del Sistema de Recuperación de Errores

El sistema implementa recuperación de errores en modo pánico a nivel de token. Cuando el analizador encuentra un carácter o secuencia inválida, ejecuta los siguientes pasos sin detener el análisis:

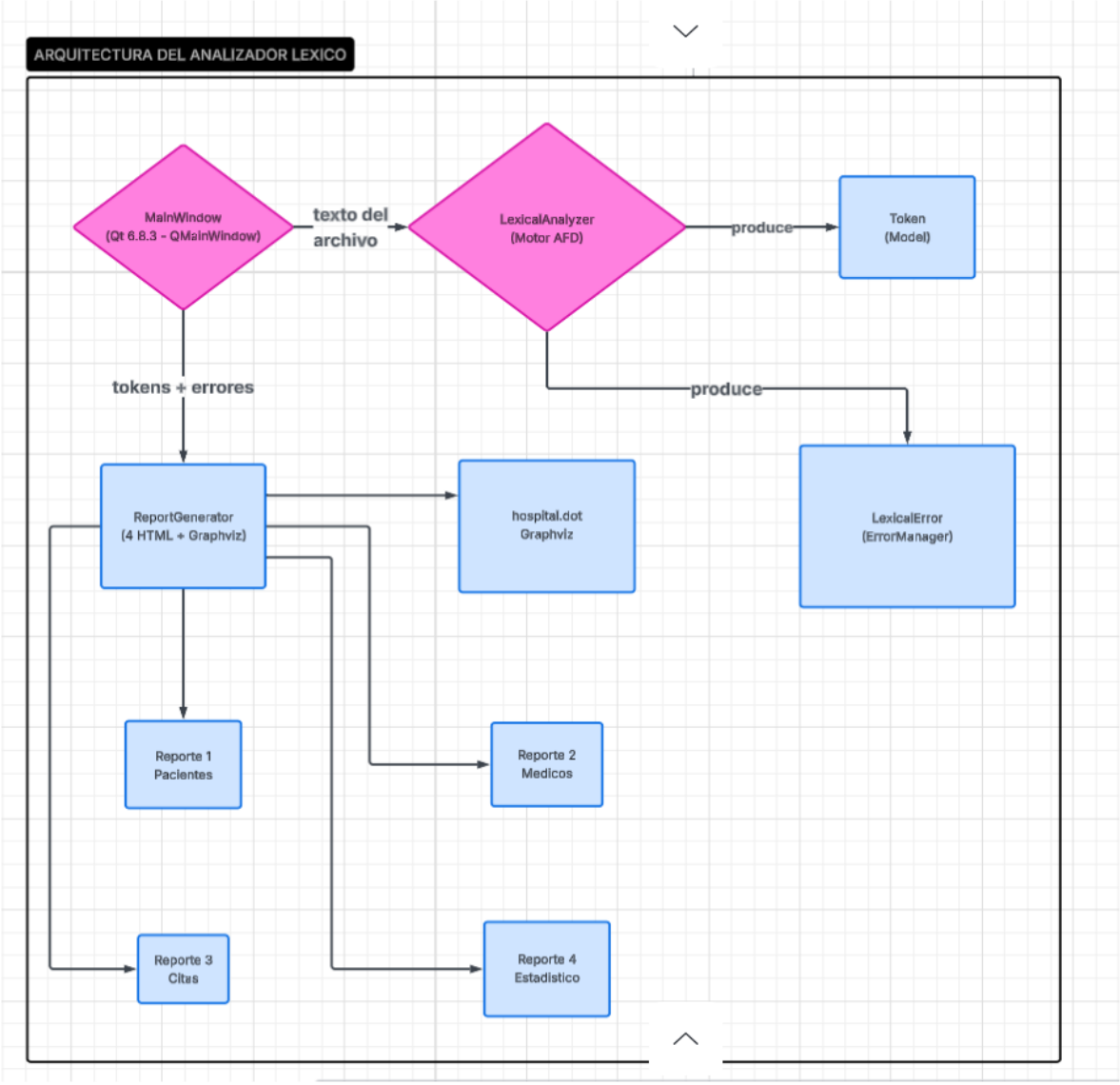
Primero registra un objeto LexicalError con el lexema exacto que causó el problema, la línea y columna precisas donde ocurrió, una descripción clara del error con el valor correcto esperado, y la gravedad del error clasificada como ERROR si el análisis puede continuar normalmente o CRÍTICO si el error impide procesar el bloque actual como en el caso de cadenas sin cerrar.

Segundo reinicia el lexema a cadena vacía y el estado a q0, lo que permite continuar el análisis desde el siguiente carácter disponible. Este comportamiento garantiza que todos los errores del archivo se detecten y reporten en una sola pasada, evitando que el usuario tenga que corregir y re-analizar el archivo múltiples veces.

La tabla de errores en la GUI muestra todos los errores acumulados al finalizar el análisis, ordenados por número de ocurrencia. El reporte HTML de errores presenta la misma información con formato visual donde los errores CRÍTICO se resaltan en rojo oscuro y los ERROR en rojo claro.

DIAGRAMAS HECHOS CON LUCID Y GRAPHIZ ONLINE:

1. Arquitectura del Sistema


ARQUITECTURA DEL ANALIZADOR LEXICO
MainWindow
(Qt 6.8.3 - QMainWindow)
texto del
archivo
LexicalAnalyzer
(Motor AFD)
produce
Token
(Model)
tokens + errores
produce
ReportGenerator
(4 HTML + Graphviz)
hospital.dot
Graphviz
LexicalError
(ErrorManager)
Reporte 1
Pacientes
Reporte 2
Medicos
Reporte 3
Citas
Reporte 4
Estadistico

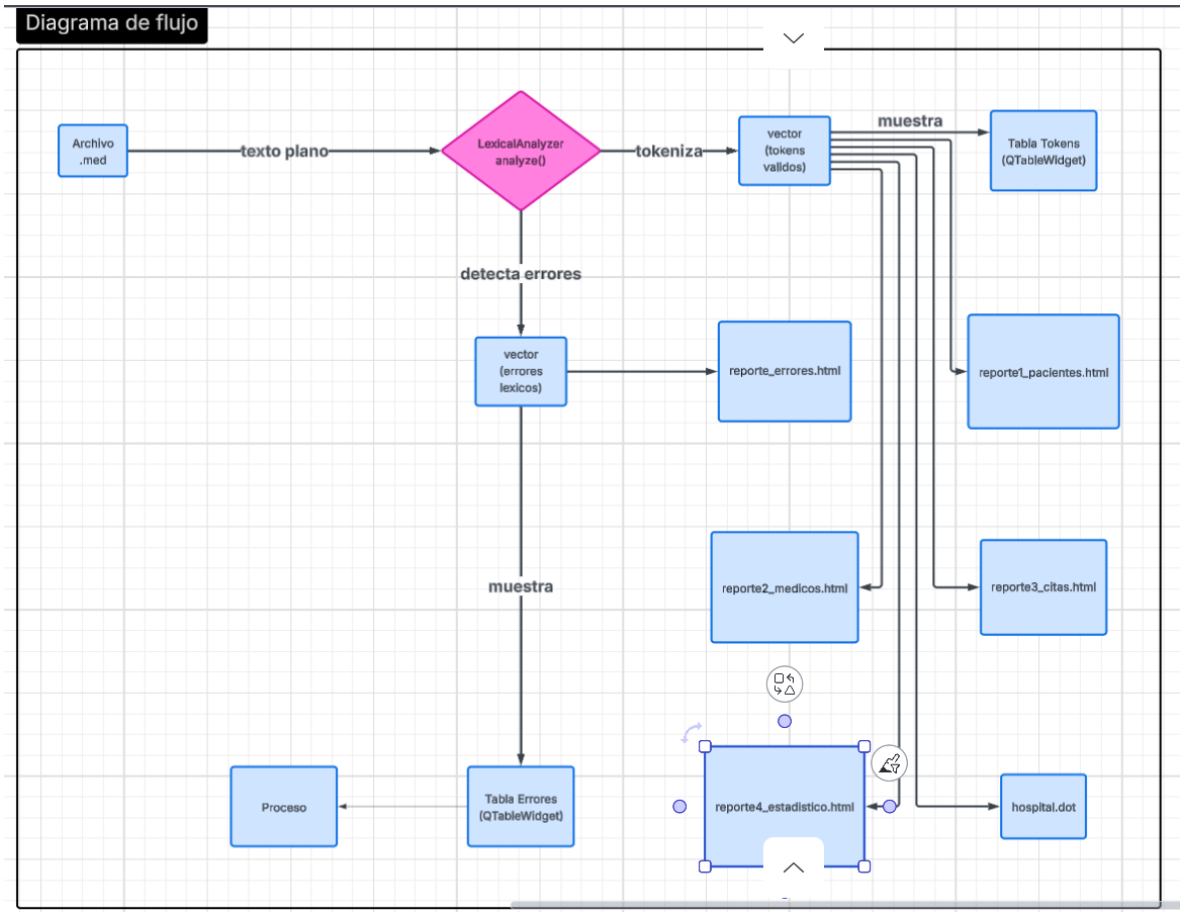
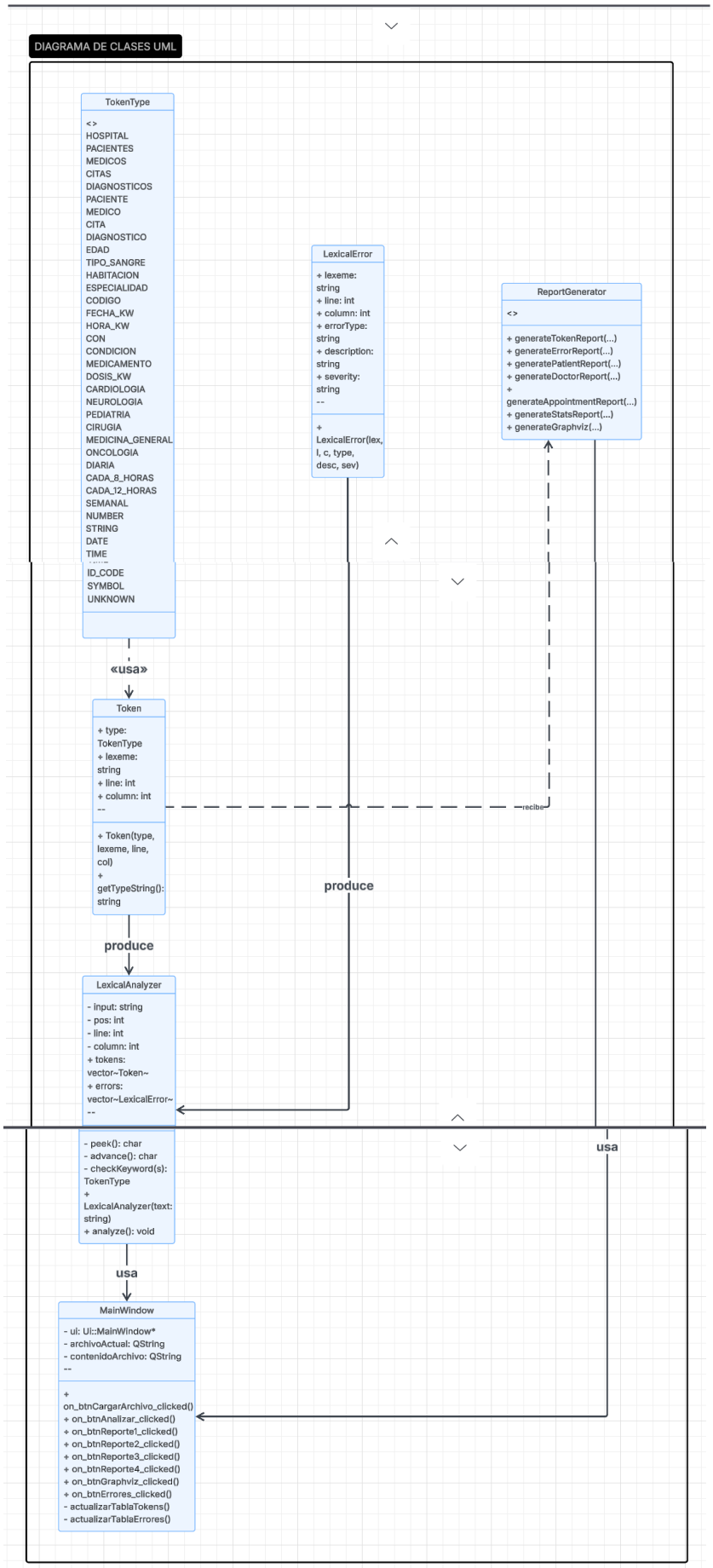
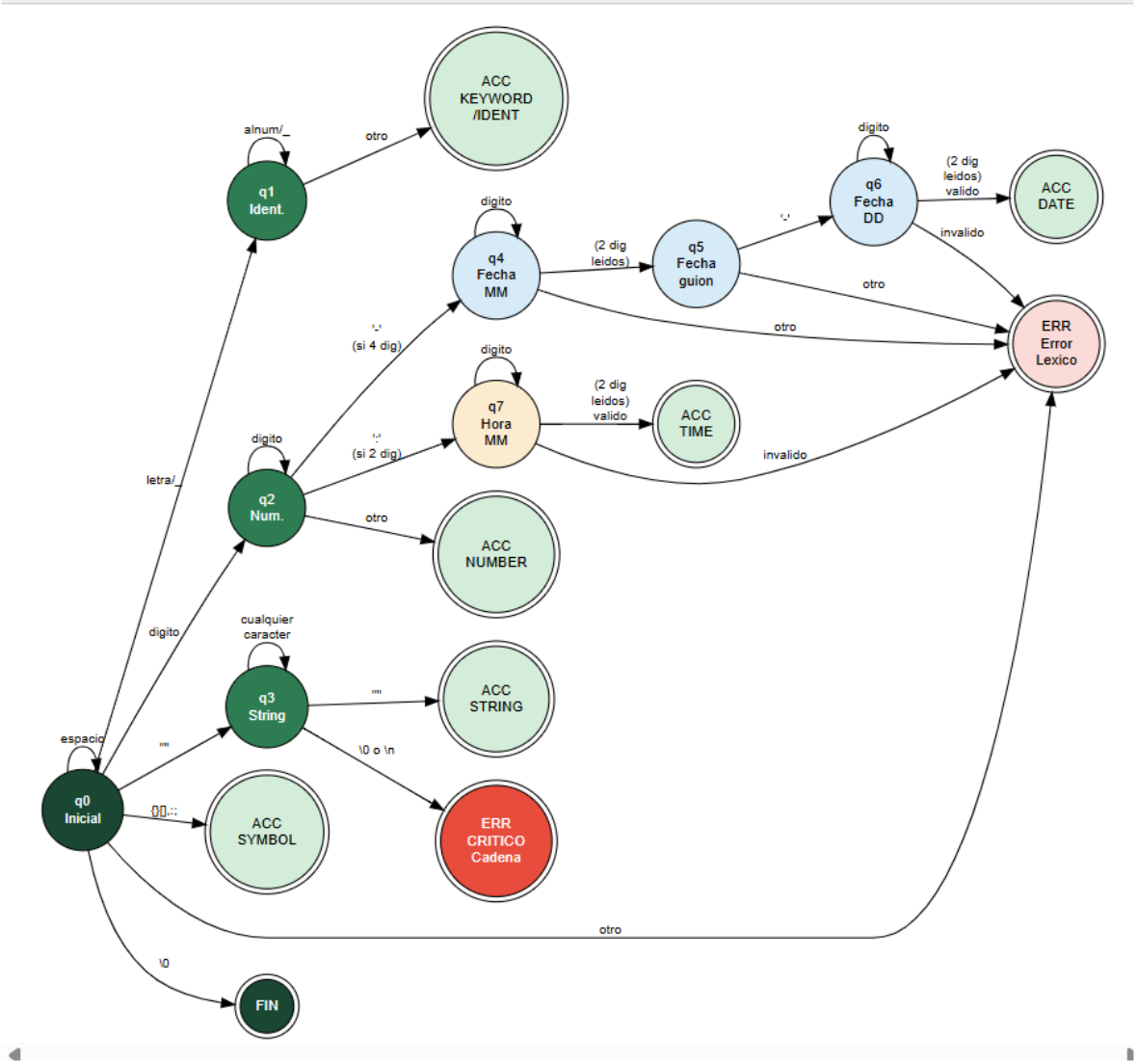


Diagrama de flujo
Archivo .med
texto plano
LexicalAnalyzer analyze()
tokeniza
vector (tokens validos)
muestra
Tabla Tokens (QTableWidget)
detecta errores
vector (errores lexicos)
reporte_errores.html
reporte1_pacientes.html
muestra
reporte2_medicos.html
reporte3_citas.html
Proceso
Tabla Errores (QTableWidget)
reporte4_estadistico.html
hospital.dot

1. Diagrama de Clases UML



2. Diagrama del AFD



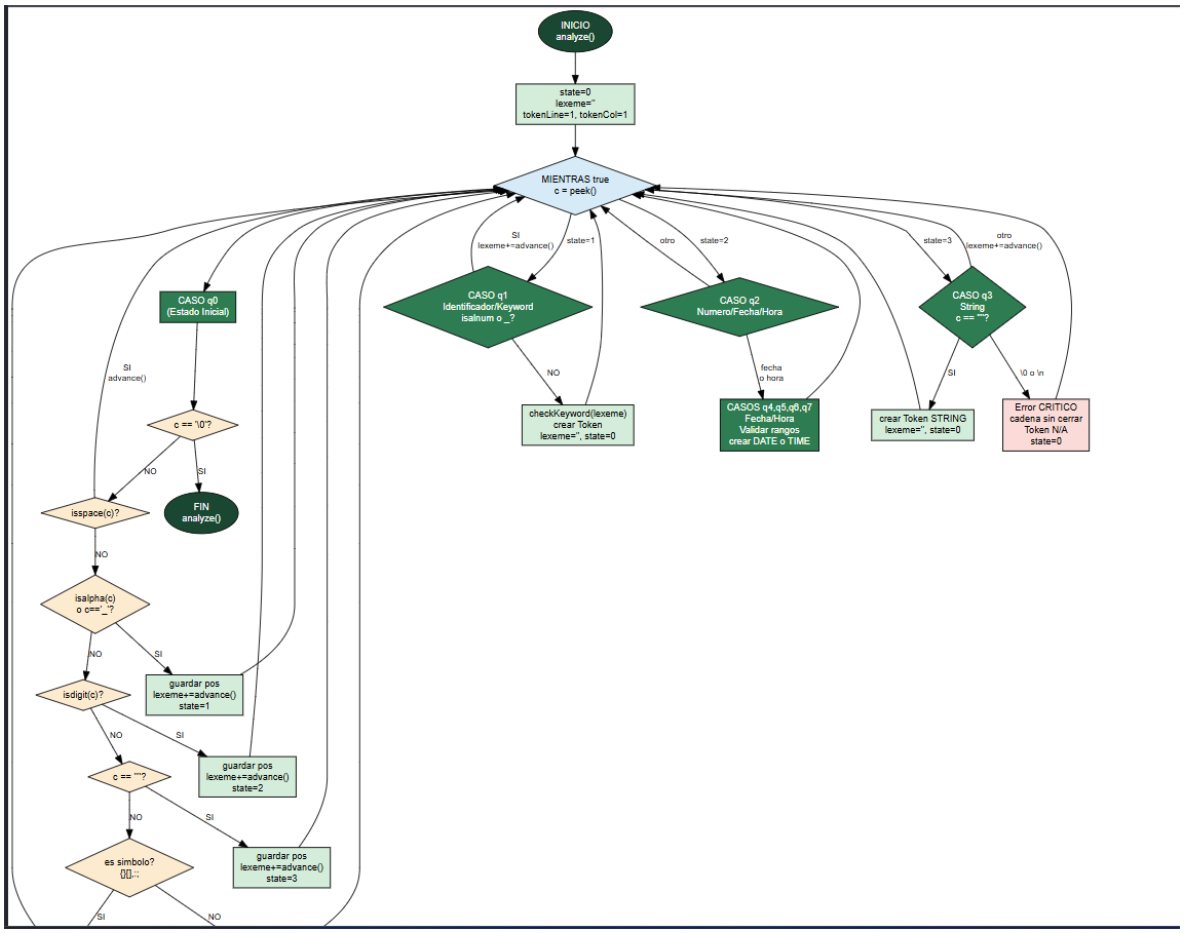
3. Tabla de Transiciones del AFD

| Estado | Letra/_ | Digito | " | - | : | {}[].,; | Espacio | \0 |
|---|---|---|---|---|---|---|---|---|
| **q0** | q1 | q2 | q3 | ERR | SYM | SYM | q0 | FIN |
| **q1** | q1 | q1 | ERR | ERR | ERR | ACC* | ACC* | ACC* |
| **q2** | ERR | q2 | ERR | q4 (4dig) | q7 (2dig) | ACC | ACC | ACC |
| **q3** | q3 | q3 | ACC | q3 | q3 | q3 | q3 | ERR-CRIT |
| **q4** | ERR | q4 | ERR | ERR | ERR | ERR | ERR | ERR |
| **q5** | ERR | ERR | ERR | q6 | ERR | ERR | ERR | ERR |
| **q6** | ERR | q6 | ERR | ERR | ERR | ACC | ACC | ACC |
| **q7** | ERR | q7 | ERR | ERR | ERR | ACC | ACC | ACC |

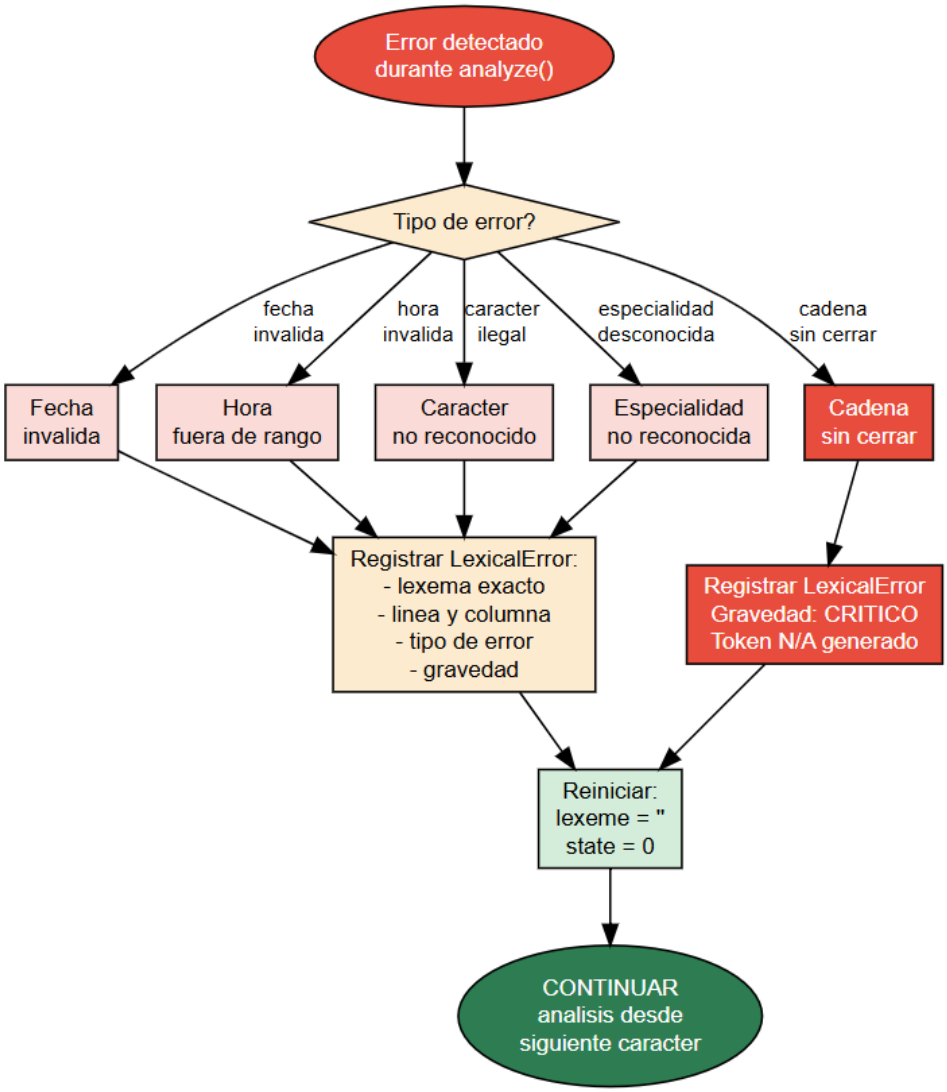
* ACC = Aceptar y hacer keyword lookup
ERR-CRIT = Error critico cadena sin cerrar
SYM = Token SYMBOL

## 4. Descripción de Algoritmos

### 4.1 Algoritmo de Tokenización

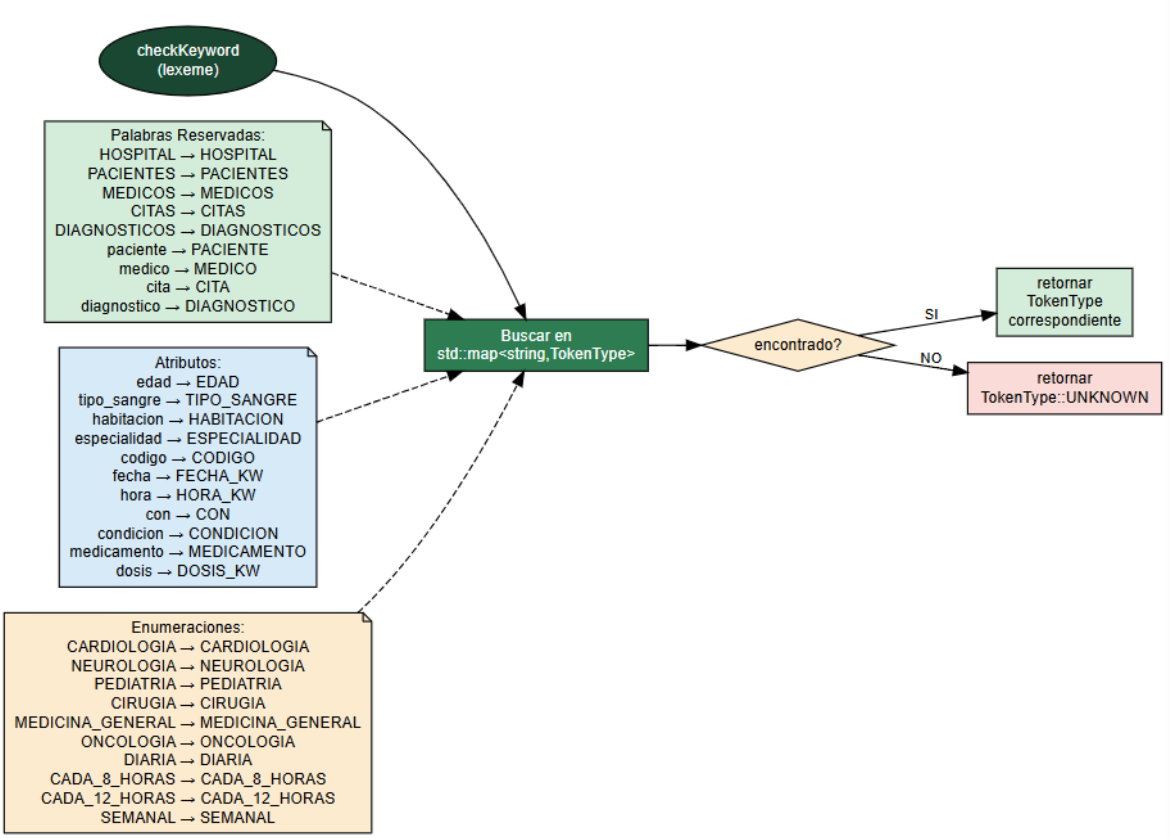


## 4.2 Algoritmo de Recuperación de Errores




El sistema implementa recuperación de errores en modo pánico a nivel de token:

## 4.3 Algoritmo de Keyword Lookup




## 6. Justificación de Decisiones de Diseño

### 6.1 AFD con switch/case en lugar de tabla explícita

Se eligió implementar el AFD usando switch(state) con estados numéricos por las siguientes razones:

- **Legibilidad:** Cada case corresponde directamente a un estado del AFD diseñado en papel, facilitando la verificación y el debugging.
- **Eficiencia:** El switch en C++ compila a una tabla de saltos, siendo O(1) en tiempo de ejecución.
- **Mantenibilidad:** Agregar nuevos estados solo requiere agregar un nuevo case sin modificar el resto del autómata.

### 6.2 Prohibición de std::regex

El enunciado prohíbe explícitamente el uso de std::regex. Esto obliga a implementar el AFD manualmente, lo que tiene ventajas reales:

- **Control total** sobre el tracking de línea y columna.
- **Mensajes de error precisos** con posición exacta.
- **Mejor rendimiento** al evitar el overhead del motor de expresiones regulares.

## 6.3 Separación en clases

El proyecto se dividió en 5 clases con responsabilidades claras siguiendo el principio de **Responsabilidad Única (SRP)**:

| Clase | Responsabilidad |
|---|---|
| Token | Representar un token con su metadata |
| LexicalError | Representar un error con su contexto |
| LexicalAnalyzer | Tokenizar el texto de entrada |
| ReportGenerator | Generar los 4 reportes HTML y Graphviz |
| MainWindow | Manejar la interfaz gráfica del usuario |

## 6.4 Reportes como HTML estático

Se eligió HTML con CSS embebido porque:

- Se abre en cualquier navegador sin instalar nada.
- Permite estilos visuales ricos con colores semánticos.
- Es portable entre sistemas operativos.

| Clase | Responsabilidad |
| --- | --- |
| • Cumple exactamente con lo que pide el enunciado. | |

**6.5 Recuperación de errores continua**

El analizador nunca se detiene ante un error léxico. Esto es fundamental porque:

- Un archivo hospitalario real puede tener errores parciales.
- El médico o administrador necesita ver TODOS los errores de una sola pasada.
- Detener en el primer error obligaría a múltiples ciclos de corrección.